Vibe Code Studio - Setup and Use

Vibe Code Studio

Turn raw talking-head footage into finished, captioned vertical clips - and make the covers to go with them - from one Claude Code setup.

Two jobs, one kit:

1. **Edit a video.** Raw footage in, captioned 9:16 clips out: transcribe, pick the best moments, cut, face-track to vertical, style captions, render. (This pipeline is built on the open-source Vibe Editing plugin - see `NOTICE.md` for credit.)
2. **Make a thumbnail.** A photo or a frame from your video plus a headline becomes a YouTube thumbnail (1280x720) and a vertical cover (1080x1920) in your brand, with platform safe zones handled for you.

What you need

- A Mac or Linux machine, with Claude Code.
- Homebrew (macOS) so setup can install ffmpeg and friends; on Linux use your package manager.
- Your own keys for anything that costs money. Nothing is uploaded anywhere; keys stay in a local file on your machine.

Honest costs

The kit itself is free and open source (MIT). Running it is not fully free:

- **Claude**: you need Claude Code access (subscription or API).
- **Transcription**: the default path uses AssemblyAI, a paid pay-as-you-go API. A local offline fallback (faster-whisper) is included if you prefer not to pay - it's slower and needs no key.
- Everything else (ffmpeg, fonts, the thumbnail builder) is free.

Quickstart

```
git clone <this repo> && cd vibe-code-studio
./setup.sh                        # installs dependencies, runs a health check
```

Then open the folder in Claude Code and say:

```
Set up Vibe Code Studio for me
```

The agent walks you through brand, keys, and a **sample project** - a synthetic test clip and test covers - so you see a real result before using your own footage.

After that:

```
Edit this video: path/to/take.mp4
Make a thumbnail for it: headline "CLAUDE EDITS MY VIDEOS", accent word EDITS
```

Or from the terminal:

```
python3 skills/thumbnail/scripts/thumb.py frame --video take.mp4 --time 4.0 --out
        frame.png
python3 skills/thumbnail/scripts/thumb.py make --image frame.png \
    --headline "CLAUDE EDITS MY VIDEOS" --accent EDITS \
```

```
--preset brand/presets/default.json --outdir covers/
```

Make it yours

Your brand is a preset file, not something you're stuck with. Copy `brand/presets/default.json`, change the colors and font, and pass it with `--preset`. The `vibe-code-ceo` preset ships as a worked example.

Before you trust it

- `sample/make_sample.sh` creates a tiny generated test clip.
- `python3 skills/thumbnail/scripts/thumb.py selftest` renders test covers and checks the exact platform sizes.
- `python3 plugins/vibe-editing/doctor.py` checks the editing pipeline's health.

Always look at the rendered cover and the edited clip before posting. Your originals are never modified: the kit only reads them and writes new files to an output folder you choose.

Test status (verified in this repo's sandbox, Linux):

- Thumbnail/cover builder: tested end-to-end, including `--cutout` background removal on a real (public-domain) photo, frame extraction from video, both brand presets, both output sizes, and the error path. Final renders visually checked.
- Caption pipeline: tested end-to-end offline - local faster-whisper transcription (no API key), text normalization, style stream, burn, and the caption/audio sync gate all ran and passed on a sample clip, in both the default word-by-word look and the phrase-chunk preset (`brand/presets/captions-phrase.json`).
- The all-in-one `spice_caption.py` wrapper stops without an AssemblyAI or Groq key; the key-free route is the documented manual chain (local whisper -> spice_format -> generate_spice), which is what was tested.
- Not yet run: a clean install on a fresh machine, macOS (upstream docs target macOS via Homebrew), the full `/edit` orchestration with silence cutting and reframing, and the `caption_sync_gate.py --clip` self-check, which expects a `whisper-cli` binary it does not ship (the `--audio-words` route works). Treat the first local setup on your Mac as the real test. Windows is untested - use WSL.

License and credit

MIT - see `LICENSE`. Built on Vibe Editing (MIT) with attribution in `NOTICE.md`.

Guided onboarding (for your Claude Code agent)

Read this whole file, then set the kit up with the user. Work in order. Ask before installing anything, before spending anything, and before touching files outside this repo. Keep all secrets local - never commit keys, never upload them.

1. Health check

Run `python3 plugins/vibe-editing/doctor.py`. Report what is missing in plain words and offer the exact install command for each missing piece (`./setup.sh` handles ffmpeg and Python dependencies on macOS via Homebrew).

2. Keys (local only)

Ask which transcription path they want:

- **AssemblyAI** (default, paid pay-as-you-go): they paste their key; save it with `python3 plugins/vibe-editing/onboard.py set ASSEMBLYAI_API_KEY`. Tell them this costs money per minute of audio.
- **Local fallback** (free, slower): no key needed; uses faster-whisper already in requirements.

Never print a key back. Never write keys anywhere except the kit's private config. Keys must never appear in git.

3. Brand preset

Ask four questions: brand name, headline color, accent color, and whether they have a preferred font. Then create `brand/presets/<their-name>.json` by copying `brand/presets/default.json` and changing those values. Show them the file. `brand/presets/vibe-code-ceo.json` is a finished example.

4. Prove it on the sample

1. `bash sample/make_sample.sh` - generates a synthetic test clip.
2. `python3 skills/thumbnail/scripts/thumb.py selftest --preset brand/presets/<their-name>.json`
   - renders test covers; open them and confirm the sizes report OK.
3. Ask them to look at both covers. Only move on when they say the look is right (adjust the preset if not).

5. First real job

Ask for one short vertical talking-head take (phone footage is fine). Run the edit pipeline per `plugins/vibe-editing/docs/ONBOARDING.md`, then make its cover with the thumbnail builder. Preview everything before they post. Their original footage is never modified.

Rules that always apply

- Originals are read-only; outputs go to a folder the user picks.
- If a step fails, say what failed and the single next command to fix it.
- Only claim support for setups you have actually tested on this machine.